

Detecting Flaky Failures in Continuous Integration using Predictive Modeling

Raphael Frank

Seminar: Software Quality

Advisor: Fabian Leinen

Technical University of Munich

`04.raphael.frank@tum.de`

Abstract. Flaky tests present a significant challenge in continuous integration (CI) environments. A software test is considered flaky if its results vary despite being executed on the same code base, causing developers to spend valuable time searching for the root cause of failed tests. This can significantly slow down the development process. This study investigates different feature sets of readily available CI pipeline information to train a flaky failure prediction model. It builds upon an existing flaky test prediction model and enhances it by incorporating information retrieval from the CI pipeline. The data set for this study was gathered from an open-source project comprising 94 non-flaky and 93 flaky test executions of distinct test cases. Various test execution metrics, test case environment attributes, and code churn indicators were used and analyzed for their impact on test flakiness. The best feature set achieved an F1 score of 95.6%. Our results demonstrate that further CI pipeline information can enhance the prediction of flaky test failures, thereby improving the efficiency of the development process.

Keywords: Flaky Tests · Flaky Failures · Prediction Model · Machine Learning

1 Introduction

Continuous Integration (CI) has become a significant part of today’s software development process [13]. It describes frequently adding code to the code base to identify integration problems as early as possible [5]. Before merging into the working branches, the code is automatically tested. Usually, tests should either pass or fail, depending on the changed code. The term *flaky tests* was first introduced by Biswas et al. [4] and describes tests that both pass and fail on the same code under test. Empirical studies have found that many tests can be flaky, depending on the project characteristics, highlighting the relative nature of flakiness [20]. Other studies have shown that flakiness in CI is a significant concern and negatively impacts the development process [11], emphasizing the need for solutions. Since its first appearance in research, different causes of flakiness have been examined and classified [23]. Research into promising practices in CI [3] and test failures in general [23] has led to a better understanding of flaky tests in the pipeline. In this context, we define a *flaky failure* as a test failure that can pass when run on the same code base. After executing a test suite, the industry is keen to identify which failures are flaky and which are not in real time. Therefore, considerable research has gone into finding effective approaches like automatic rerunning of failed test cases, or DeFlaker [3], a coverage-based approach.

However, these models and solutions are often restricted in their use cases. Collecting numerous coverage data in a small Java project may be suitable but definitely not applicable to large multi-language projects. Coverage also no longer works for large system tests, which are far more prone to flakiness [15]. Other solutions like iDFlakies [17] are trying to only identify flaky tests. In practice, however, we are more interested in identifying flaky failures since we

probably already know which tests are flaky. Flaky failures are the ones that slow down the development process the most. Overall, current research is satisfied with specific use cases rather than applicable solutions. A recent study by Gruber et al. [14] used a more general approach, utilizing test history data and code evolution. Further research [12] suggests that additional information retrieval from the CI pipeline can help identifying flaky failures.

This study aims to investigate this assumption by evaluating different feature sets and their impact on a flaky failure prediction model. We gathered test results from the CI pipeline of the open-source Microsoft Playwright repository¹. The dataset includes 4,563,149 test case executions across 259 commits. Focusing on potentially flakily failed tests, we analyzed 94 non-flaky and 93 flaky test failures of different test cases. Additional information extended the dataset, allowing us to evaluate different feature sets using supervised machine learning. We categorized the different features to analyze them for their impact.

Our results suggest that the fail rate paired with features representing a test-file correlation leads our model to perform very well in predicting flaky failures. At the same time, the test case environment and basic code churn information are less impactful. The best features combined achieved an F1 score of 95.6%. This result indicates that readily available information in the CI pipeline can train an effective general model capable of predicting flaky failures better than the standard method of instantly rerunning tests. Keeping the information as general as possible ensures that projects of any complexity can benefit from flaky failure prediction.

2 Related Work

Since the first extensive study about flaky tests by Luo et al. [20], the topic has gained significance both in research and the industry [21]. Until today, hundreds of studies have been published covering various aspects of flakiness. However, most of them do not include flaky failures in CI, although they are very relevant in practice. Since flaky tests are not always obvious, detecting them and investigating the root causes of flakiness has become an essential part of the research. We classify related research into three categories, each following a different approach.

2.1 Root Cause Investigation

Many studies empirically research the root causes of flakiness to gain deeper insights into flaky tests and their occurrences in real-life applications. Luo et al. [20] already briefly classified the root causes. Parry et al. tried to pinpoint which causes mainly occur in practice [21]. They find asynchronous waiting and concurrency to be the most prevalent. Overall, these studies provide a better understanding of flaky behavior by investigating the underlying reasons.

Theoretical research often lacks suggestions for practical usage. However, we can use the extensive understanding of the root causes to create features for our model. This way, we can transform the theoretical research into practical approaches.

2.2 Dynamic Analysis

Dynamic analysis heavily relies on instant feedback from the CI pipeline and the underlying system. Since they try to solve practical issues, practitioners and researchers have worked on tools that dynamically detect flaky tests in the project. DeFlaker [3] by Bell et al. automatically detects flaky failures in software systems without the need to change the source code. Monitoring previous correlations between tests and code changes analyzes the historical results of test executions to identify patterns that point to flakiness. Another dynamic analysis tool focusing on

¹ <https://github.com/microsoft/playwright>

test order dependencies is iDFlakies [17] by Lam et al. This tool analyses test order dependencies and their underlying flakiness to classify them.

These tools are practical because they rely on something other than a vast database. Since they work in real-time, the flakiness added to the system is usually detected after a short period. The downside of most of these approaches is that they are often limited to specific project requirements like the programming language or the testing framework. Developing a reliable dynamic analysis tool with as few limitations as possible can be complicated. When working on a dynamic approach encapsulated directly into the CI pipeline, these limitations often come.

2.3 Predictive Modeling

Predictive models use machine learning and statistical techniques to predict flaky tests based on historical data and various metrics. These models analyze patterns in previous CI data to predict which tests are prone to have flaky behavior. Although there are some approaches in this field, like FlakeFlagger [1] by Alshammari et al. that uses test characteristics or FLASH [10] by Dutta et al. that provides a heuristic approach by systematically rerunning tests on different random number generators.

As stated before, there are some approaches to detect flaky tests by using machine learning on CI data. However, in most cases, they rely on more insight into the project, like the code itself or environment parameters, to retrieve more data. This limits the possible use cases of these models, especially on larger and more complex systems, since not all projects may be able to provide this further information. Overall, we see a lack of more general approaches that do not rely on more specific data.

3 Approach

As we have seen, there is a gap between what most research presents and what would be the most practical for the industry. Gruber et al. [14] took the first step into a more general approach. We now want to build upon that, including more complex and hidden data, while working only with readily available information in the CI pipeline. This information should be independent of any programming language, the system, or the project environment so that this approach can be integrated into most processes as possible. We now discuss the different steps we took to implement the model. An overview of the model can be seen in Fig. 1.

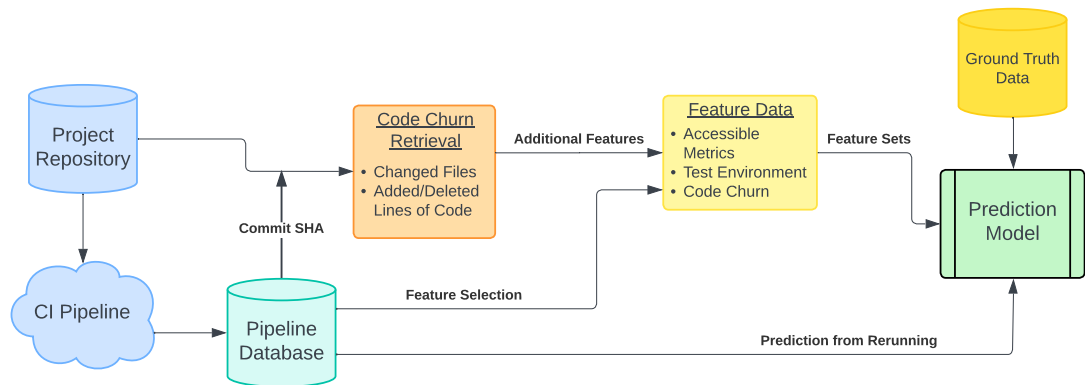


Fig. 1: Overview of our methodology. We work on a given database, extend the data, and then train the model with the different features to get the results.

To train a prediction model, we need data that can be a predictor for flaky failures. We call these a *feature* for our model. We use supervised machine learning with two selected classifiers and cross-validation with five folds to evaluate. We interpret the results using common precision, recall, and F1 score metrics [6]. It turned out that support vector machines and random forests are meaningful classifiers. A support vector machine (SVM) is an algorithm that uses mostly independent data (support vectors) to create a hyperbolic plane for prediction [9]. The random forest classifier combines multiple decision trees to form a forest [7].

To find a selection that yields the highest f1 score, we watch each feature’s impact on our model, aka the SHAP values. They are a measurement for the impact of features to the model [19]. Because we can only calculate the values of one trained model and we want to measure the impact of every feature, we train a model with all the features we have. Our approach is to select more features systematically, starting with the ones with the highest SHAP value since a high SHAP value indicates a substantial impact on the model. This way, we can systematically create a model that performs best.

3.1 Feature Categories

Features can be different in nature, but the data is specific for each test execution. Gruber’s research [14] included a linear commit history on one branch. Therefore, he was able to make use of features that depend on recent linear changes in the code. We worked on a dataset that consists of commit data from different branches. That means we had to select features that do not depend on linearity or commit history in the same branch but on the test nature itself.

To compare features of different types and availability, we categorize them into three groups. All features were processed using numerical data since machine learning algorithms can only work with numerical data. An overview of the features can be found in Table 1.

Table 1: Overview of the different feature categories.

Accessible Metrics	Test Case Environment	Code Churn
• fail rate • test duration • mean pass duration • mean fail duration	• operating system • web browser	• changed files • changed extensions • added lines of code • deleted lines of code • test-file history • test-file similarity

Accessible Metrics We categorize information that is mostly available without more effort in this category. Unlike other approaches that use the flip rate or entropy [15] of the test outcomes, we had to adapt to the nonlinearity of our data. We used the overall fail rate [11] specific to a test as a metric. The fail rate r_f is defined by the ratio of failed test outcomes n_f to its sum with passed test outcomes n_p ($\sim$ number of test executions) as follows:

$$r_f = \frac{n_f}{n_f + n_p} \quad (1)$$

We only count the initial test result since reruns would not occur in our model. Since we only considered distinct test cases when training our model (see 3.2), we calculated the fail rate for the most recent test execution. This way, we could use this test’s more extensive commit history, even if the commit history itself is not linear. For the fail rate, we expect some flaky

tests to fail more often than others [11]. However, we expect flaky tests to fail more often than regular ones since completely broken tests are usually fixed relatively quickly.

The other generally accessible metric in this category is the test duration [12]. We consider each test’s duration and its mean pass and fail duration. To compute the mean pass duration $\bar{t}_p$, we calculate the sum of n passed test durations t_i as the following:

$$\bar{t}_p = \frac{1}{n} \sum_{i=1}^n t_i \quad (2)$$

The mean fail duration is computed accordingly. As described in Luo’s research [20], one of the causes of flakiness in regression testing is due to asynchronous waiting or waiting for an external service in general. Accordingly, we expect flaky tests to take significantly longer than simple failed tests. Again, this does not have to be a characteristic of all flaky failures, but we expect at least some correlation. Additionally, a test duration with a particular verdict would be expected to be near its mean duration specific to the verdict. So, for example, we would see a passed test take just about as much as its mean pass duration and the same for its failing counterpart.

Since we expect the test outcomes and duration to be constantly available, we assigned them to the general category. Additionally, we expect them both to be reliable indicators because of their simplicity.

Test Case Environment The data set we worked with also included information about the overall testing environment the test was running on. We are interested in the operating system and the web browser. Both are interesting to us because they might affect the underlying execution of the code. The project consists of a web testing framework, and the web browser might, therefore, also have an impact. Collecting the information was rather difficult since the values were specific test environments like `chromium-ubuntu-22.04-node20`, `smoke`. We extracted the operating system and the used web browser from these values by searching if the string includes the specific OS or browser name. The web browser is relevant because the project we get the data from is a web framework for testing. The tests were executed on Windows, MacOS, and Ubuntu for the operating system and Chromium, Firefox, and Webkit for the web browser. For the OS and browser, one can find an overview of each feature in Fig. 2 and Fig. 3. It is noticeable that most of the tests are running on an Ubuntu-Chromium combination. Anwyl-Irvine et al. already investigated the performance of software on different platforms [2], so the performance and relevance of this combination could be a reason for this. As said earlier, we had to convert the enumeration-based feature into a numerical vector using one hot encoding [8].

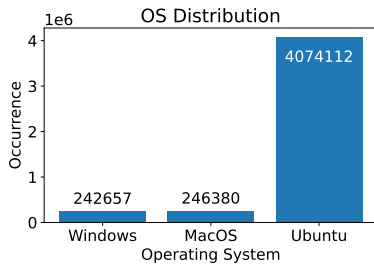


Fig. 2: Distribution of operating systems in the test environment.

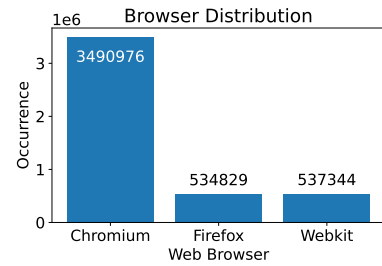


Fig. 3: Distribution of web browsers in the test environment.

We include these features to investigate how much influence the testing environment has over the flakiness of the test. Other studies have already suggested that an unstable environment might cause flakiness [22]. Primarily, we wanted to see if the test environment might be a significant indicator in predicting flaky failures.

Code Churn To extend the already available features with more complex ones, we included more specific information about the commit and what was changed. This information about the code change is usually referred to as *code churn information* [21]. We included the number of changed files, the number of different types of files that were changed (by extension), and the number of changed lines of code (LoC). The common belief is that more modifications to the code generally lead to more changes in the program characteristics and, therefore, failure potential [11]. Therefore, we expect the number of changes in different forms to impact the failure prediction.

The work of Elsner et al. [12] about regression test optimization also included features about the test-file history and similarity. They also seem to have a noticeable impact on the flakiness of tests since they represent associations between a test and the code it is testing when executed. Normally, we would expect a before-passing test to only fail if the code (or the files) it is testing was changed. The test-file history metric measures the frequency of a file being changed on the commit where a test fails [12]. Therefore we calculate the test-file history frequency $f_{(t,f)}$, where (t, f) is the pair of test t and file f , as follows:

$$f_{(t,f)} = \frac{n_f}{n} \quad (3)$$

We consider n as the total number of fails this test had, and n_f the number of fails this test had where file f was changed in the commit. The other new metric is the test-file similarity. We compare the test name as well as the file name lexicographically. For this, we calculate the Levenshtein distance [18] like Elsner [12] did. It is a measurement of the lexicographical distance between two strings. Intuitively, the Levenshtein distance counts the number of character changes (insertions and deletions) necessary to convert one string to another. Since all file names are of different lengths, we normalize the distance by dividing by the greater length of both strings. We also want the value for tests and files that are closely connected to be near 1 while independent pairs are near 0. We achieve this by subtracting our normalized distance from 1. With all together, we get the following equation for our feature f , where $d(s_1, s_2)$ describes the Levenshtein distance between string s_1 and s_2 :

$$f = 1 - \frac{d(s_1, s_2)}{\max\{|s_1|, |s_2|\}} \quad (4)$$

Overall, the test-file history metric is more logical, while common developing practices strengthen the similarity metric. Most research in this field also includes the number of authors in the commit. Since our commit data always has only one author, we did not include this metric as an additional feature.

We have a mapping from the test-file pair to the calculated value for both the test-file history and similarity. For the feature itself, we only want one value, and since one strong correlation between a file and a test (regardless of the specific metric) is enough for a bug to let a test fail, we consider the maximum value of each.

3.2 Data Set and Code Churn Retrieval

From the 4.563.149 test executions, we filter out data that contains invalid data due to fractals in the retrieving process or the original data set. However, this only affects a neglectful proportion

of passing tests and can be ignored. To ensure that the model does not suffer from data bias, we only consider distinct test cases, randomly sampled, from the 1.302 non-passing test executions. This data bias is generally referred to as *overfitting* [24]. It can negatively affect our evaluation. After the selection, we have 94 non-flaky and 93 flaky test executions according to our ground truth data [16].

The data set then includes basic metrics like the test case name, the commit SHA, and the test result. At this stage, we could drop any unneeded information like the job ID or the branch name. Some general information, like the test result or the duration, is already available to evaluate, while others have to be calculated or retrieved in another way.

In the database, the verdict data of each test execution is either passed or failed since the pipeline only considers those two. To determine if a test is flaky, we calculate the sum of passed and failed executions in one job, and if both are greater than 0, we consider the test execution flaky for the standard prediction. On the other hand, the database did not include more specific code churn information. We, therefore, collected the number of changed lines of code and file names by finding the specific commit on GitHub using only the commit SHA. Unfortunately, the database consists of commits outside of any branch as individual forks and are, therefore, unavailable by forking the repository. After we collected this information, we could assign them to the features. We had to connect each test-file pair with its individual pass history to calculate the test-file similarity and the test-file history. Although this metric could suffer from the independence of the test cases and the history, we believe this will not have this much impact because the connectivity between a test and a specific file usually does not change much between commits. We proceeded with the before-stated equations to calculate the metrics for the rest of the data and completed our dataset.

4 Evaluation

In practice, flaky tests that pass are often not examined any further. Instead, flaky failures consume most of the development time assigned to investigating flakiness. For that reason, we focused our model on predicting only flaky failures. We collected these features to evaluate a new approach of using readily available data in CI combined with information retrieval based on this data for flaky failure prediction. To evaluate them correctly, we want to answer these research questions:

- **RQ 1:** How well do **accessible metrics** features perform in a flaky failure prediction model?
- **RQ 2:** How well do **test case environment** features perform in a flaky failure prediction model?
- **RQ 3:** How well do **code churn** features perform in a flaky failure prediction model?
- **RQ 4:** Which **combination of features** from accessible metrics, test case environment, and code churn categories yields the highest F1 score in predicting flaky failures?

Answering research questions 1 to 3 helps us to further understand the impact specific CI metrics have on flakiness prediction models in a theoretical sense. Answering the last research question gives us a concrete model for practical flaky failure prediction that enhances the standard rerunning method and gives us an understanding of how to improve such systems in the future.

4.1 Common Flaky Prediction and Ground Truth

We are working on an already existing database. More specifically, we work with 4.563.149 test execution results of 259 commits. These results were gathered over one month from the CI pipeline of the open-source Microsoft Playwright repository from jobs that included artifacts.

Passed tests are accepted as they are and considered passed. In the given pipeline, the standard method to detect flaky tests is to instantly rerun failed tests. If all retries fail, we consider the overall verdict as failed. However, if one of the retries passes, the overall verdict is considered flaky since it has different results on the same code under test. In Fig. 4, the predicted verdict distribution is shown. One can see that only a tiny proportion of verdicts did not pass. These are the only test outcomes we could use for our analysis. Since we compare our prediction model with the rerunning method to see if it is an overall improvement, we need a ground truth to compare both. Our model should be used on failed tests to predict whether the failure was flaky. Because of this, our ground truth must only check whether the tests that also failed on the retries are flaky. Since we only use our model on failed tests, caring for them and not for passed tests is sufficient. For this, we use the rerunning data of Kratzel [16] since he worked with the same database on precisely that. Rerunning after a particular time, like on weekends, has effectively detected flaky failures for some causes of flakiness like an external state or unstable connectivity [22]. We assume we can create a model that can give an accurate prediction earlier without waiting for later rerunning.

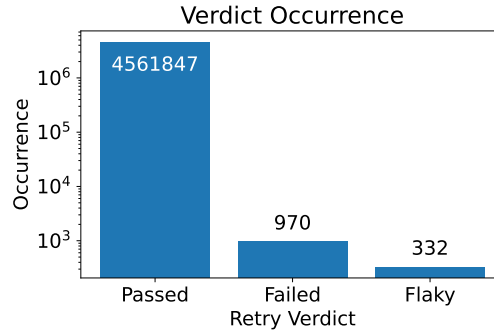


Fig. 4: Logarithmic distribution of verdicts based on simple rerunning.

4.2 Threats to Validity

External and internal circumstances could falsify the results. At first, we are working on a single open-source project. Like in many industrial research, this could lead to biased data or data that is too specific [21]. On the other hand, the focus on one database only enabled us to utilize more complex features instead of focusing too much on retrieving the data of each project. There were other available data sets from open-source repositories. However, these data sets often only include some of the data we need to implement all of our features into the model. The small amount of data might also contribute to inconclusive results. Although, even a relatively small amount of data should have at least some significance. To enhance this, we could extend our data set even further.

Also, the nonlinearity of our commit data could badly influence the validity of our feature performance since some rely on the nature of the test. In other words, if one test case differs immensely between two different commits, some features might underperform. However, in practice, these specifications are typical for continuous integration, where developers program on many branches simultaneously.

Lastly, we use the ground truth data from Kratzel [16] to validate our predictions. Failed tests are executed again at different time intervals to see if they might pass and, therefore, are flaky failures. This method might not be enough to detect all flaky failures, and the ground

truth data might have some false negatives. However, our results and most studies [21] have shown that this method already detects a significant proportion of flaky tests not predicted by the standard rerunning method. Therefore, we consider the ground truth data sufficient to validate our model.

4.3 Results

To evaluate our approach, we use the common prediction metrics of accuracy, precision, recall, and f1 score [6]. This way, we can compare our results easily to those of other studies. When not explicitly stated, we refer all results to the random forest classifier since it performed with almost all features better than the support vector machine. The support vector machine only achieved a mean F1 score of 56.1%. Regardless, it could make better use of data, such as the test environment, which the other classifier could not utilize, and it achieved good metrics that we can compare with the random forest classifier. The random forest classifier was the best-performing classifier, achieving a mean F1 score of 87.5%.

Baseline: Rerunning Prediction When a test fails, it is executed again three times. If it passes once during the rerunning, the baseline prediction considers this test execution flaky. Otherwise not. We can compare this against the ground truth data to get the metrics in Fig. 5. We can see that the precision of this approach lies by 100%. This is because our ground truth is only rerunning tests considered failed, and we have no false positives. Overall, our baseline approach achieves a f1 score of 55.8% and a recall of only 38.7%. Since most studies [20] suggest that instant rerunning is insufficient for external services or asynchronous waiting, the result is reasonably expected.

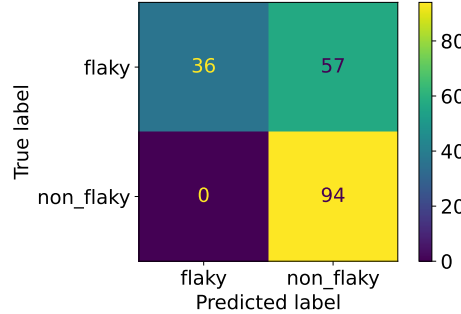


Fig. 5: Confusion matrix of the baseline prediction by simple rerunning.

RQ 1: Accessible Metrics Table 2 shows the results of training the model with readily available information. We kept the duration-related features together for this since we firmly expected a correlation between them. The fail rate achieved an f1 score of 91%, while the duration metrics performed with precision, recall, and f1 around 70%. Both sets together improved the precision to 95.1%.

Unexpectedly, the fail rate alone is far better than the baseline prediction. This strong predictor could be specific to this project or a generally good metric for non-linear test executions, even not at this intensity. The duration metrics also performed quite well, exceeding the baseline overall. We assume that the increase in precision but not recall of the combined set (fail

rate + duration metrics) comes from duration only being relevant to flaky failures rather than regular failures. This could be when some flaky tests have an unusually long duration, and some fail on average time. Since asynchronous waiting is only one of many causes for flakiness stated by studies [21], this result is reasonable. Overall, the most accessible metrics seem to make a suitable information base upon which to build.

Table 2: Results of training the model with accessible metrics. The duration set consists of test duration, mean pass duration, and mean fail duration.

Feature Set	Precision	Recall	F1 Score
• fail rate	92.5%	90.4%	91%
• duration set	73%	71.2%	71.2%
• fail rate + duration set	95.1%	85%	89.5%

RQ 2: Test Case Environment The performance of the model trained with test case environment data is seen in Table 3. Regarding precision, all combinations achieve around 70%, which is quite good compared to 18.3% and 55.7% recall. Significantly, the operating system achieves an f1 score of 27.9%.

The overall precision is rather good, compared to the recall. Again, we assume that the environment might stand out on flaky tests, but only in some cases, so detecting a flaky failure purely on the environment might be difficult. However, the operating system achieves only a bad f1 score of 27.9%, which is by far the worst performance for any feature. It points towards overall independence between flaky failures and the operating system, suggesting that flakiness occurs on all operating systems with no bias. It stands out that the support vector machine classifier led to better or equal f1 scores. This overperformance of support vector machines only occurs in this category and may be caused by the one-hot encoding of the features. Since the test cases can only adopt nine different configurations in this category, this small value range might diminish the significance of the category. This assumption is strengthened by the monotony of the data seen in Fig. 2 and Fig. 3. Although the overall performance is slightly above the baseline, test case environment information is very characteristic of a project and is, therefore, better at detecting flakiness in the system rather than a great feature in a prediction model.

Table 3: Results of training the model with test environment data. F1 SVM contains the f1 score when using the support vector machine classifier.

Feature Set	Precision	Recall	F1 Score	F1 SVM
• operating system	61.8%	18.3%	27.9%	34.4%
• web browser	76.8%	55.7%	64%	64%
• operating system + web browser	72.6%	74.1%	73.2%	73.2%

RQ 3: Code Churn Table 4 shows the results of training the model with code churn information. Although the test-file history and similarity perform alone, below the baseline and together slightly above 64.5%, we see a massive difference in precision and recall on the test-file history while the similarity is somewhat balanced. Conversely, the overall file change information performs significantly over the baseline, achieving a f1 score of 87.3%.

The vast difference in precision and recall on the test-file history indicates that the test-file history feature might capture specific aspects of flaky failures that are highly informative but sparse, leading to high precision and low recall. In contrast, the similarity feature provides a more general signal that, while less precise, captures a broader range of instances consistently. Therefore, the test-file history is suitable for detecting certain types, while the similarity points to a more general coverage. The excellent performance of the file change information confirms Gruber’s results [14] that general code churn performs great in a flaky prediction model.

Table 4: Results of training the model with code churn data. The file changes set consists of the number of changed files/extensions and added/deleted lines of code.

Feature Set	Precision	Recall	F1 Score
• file changes	92.6%	82.8%	87.3%
• test-file history	85%	28.8%	42.4%
• test-file similarity	53.6%	52.5%	52.3%
• test-file history + similarity	65.7%	64.3%	64.5%
• file changes + test-file history + similarity	79.9%	78.3%	78.7%

RQ 4: Feature Selection The model with all features performs very well with a precision of 95.3%, a recall of 93.5%, and a f1 score of 94.2%. Fig. 6 holds the calculated SHAP values of the features as a bee plot.

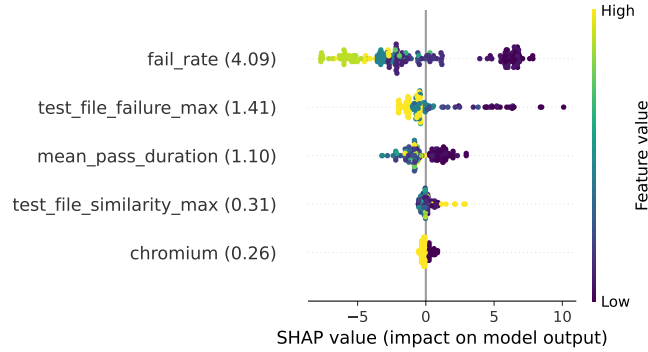


Fig. 6: SHAP values of the features on a model trained with all feature sets.

As we expected in RQ 1, the feature with the most impact is, by far, the fail rate. Against our expectations, the SHAP values indicate that a low fail rate of a test is a strong predictor of flakiness. This behavior could mean that flaky tests in our project pass most of the time and fail only under specific conditions. This result confirms that flakiness occurs in sporadic bursts rather than continuous failure. On the other hand, a low test-file history (*test_file_failure_max*) seems to be a strong predictor for flakiness like our assumption. It means that a failure of a test is probably flaky if the changed files usually do not correlate with this test.

After selecting the top 7 features according to their SHAP values, we peak with a model with an f1 score of 95.6%. An overview of its performance can be found in Table 5. This approach

can be used in similar projects to create a robust flaky failure prediction model adapted to one's needs.

Table 5: Results of training the model with SHAP-based selected features. Chromium is the binary value of the one-hot encoding of the web browser.

Selected Feature	Precision	Recall	F1 Score
• fail rate • test-file history • test-file similarity • mean pass duration • chromium • deleted lines of code • test duration	96.8%	94.6%	95.6%

5 Conclusion

In the overall research on flaky detection methods, we found that most approaches only cover specific use cases without being able to adapt to a large variety of projects. We build upon an existing model to close that gap. By using further information, we were able to extend an existing database of widely available information from the CI pipeline. We then used this extended data to train a model for predicting flaky failures without the need of rerunning. By SHAP based feature selection, we were able to derive a model achieving an f1 score of 95.6%. The results showed that only working with readily available information, we can improve the data set to be practically used in such a prediction model. Future work should investigate flakiness prediction approaches that are not tight to specific use cases to support the developing process.

References

1. Alshammari, A., Morris, C., Hilton, M., Bell, J.: Flakeflagger: Predicting flakiness without rerunning tests. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE). pp. 1572–1584 (2021). <https://doi.org/10.1109/ICSE43902.2021.00140>
2. Anwyl-Irvine, A., Dalmaijer, E.S., Hodges, N., Evershed, J.K.: Realistic precision and accuracy of online experiment platforms, web browsers, and devices. *Behavior research methods* **53**(4), 1407–1425 (2021)
3. Bell, J., Legunsen, O., Hilton, M., Eloussi, L., Yung, T., Marinov, D.: Deflaker: automatically detecting flaky tests. In: Proceedings of the 40th International Conference on Software Engineering. p. 433–444. ICSE '18, Association for Computing Machinery, New York, NY, USA (2018). <https://doi.org/10.1145/3180155.3180164>, <https://doi.org/10.1145/3180155.3180164>
4. Biswas, S., Blanton, R.: Test compaction for mixed-signal circuits using pass-fail test data. In: 26th IEEE VLSI Test Symposium (vts 2008). pp. 299–308 (2008). <https://doi.org/10.1109/VTS.2008.35>
5. Booch, G.: Object-oriented analysis and design with applications (2nd ed.). Benjamin-Cummings Publishing Co., Inc., USA (1993)
6. Bradley, A.P.: The use of the area under the roc curve in the evaluation of machine learning algorithms. *Pattern recognition* **30**(7), 1145–1159 (1997)
7. Breiman, L.: Random forests. *Machine learning* **45**, 5–32 (2001)
8. Cerda, P., Varoquaux, G., Kégl, B.: Similarity encoding for learning with dirty categorical variables. *Machine Learning* **107**(8), 1477–1494 (2018)
9. Cortes, C., Vapnik, V.: Support-vector networks. *Machine learning* **20**, 273–297 (1995)

10. Dutta, S., Shi, A., Choudhary, R., Zhang, Z., Jain, A., Misailovic, S.: Detecting flaky tests in probabilistic and machine learning applications. In: Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis. p. 211–224. ISSTA 2020, Association for Computing Machinery, New York, NY, USA (2020). <https://doi.org/10.1145/3395363.3397366>
11. Eck, M., Palomba, F., Castelluccio, M., Bacchelli, A.: Understanding flaky tests: The developer’s perspective. In: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 830–840 (2019)
12. Elsner, D., Hauer, F., Pretschner, A., Reimer, S.: Empirically evaluating readily available information for regression test optimization in continuous integration. In: Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis. pp. 491–504 (2021)
13. Fowler, M., Foemmel, M.: Continuous integration (2006)
14. Gruber, M., Heine, M., Oster, N., Philippsen, M., Fraser, G.: Practical flaky test prediction using common code evolution and test history data (2023), <https://arxiv.org/abs/2302.09330>
15. Kowalczyk, E., Nair, K., Gao, Z., Silberstein, L., Long, T., Memon, A.: Modeling and ranking flaky tests at apple. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Software Engineering in Practice. p. 110–119. ICSE-SEIP ’20, Association for Computing Machinery, New York, NY, USA (2020). <https://doi.org/10.1145/3377813.3381370>
16. Kratzel, L.: How effective is rerunning flaky tests in ci?
17. Lam, W., Oei, R., Shi, A., Marinov, D., Xie, T.: idflakies: A framework for detecting and partially classifying flaky tests. In: 2019 12th IEEE Conference on Software Testing, Validation and Verification (ICST). pp. 312–322 (2019). <https://doi.org/10.1109/ICST.2019.00038>
18. Levenshtein, V.I., et al.: Binary codes capable of correcting deletions, insertions, and reversals. In: Soviet physics doklady. vol. 10, pp. 707–710. Soviet Union (1966)
19. Lundberg, S.M., Lee, S.I.: A unified approach to interpreting model predictions. Advances in neural information processing systems **30** (2017)
20. Luo, Q., Hariri, F., Eloussi, L., Marinov, D.: An empirical analysis of flaky tests. In: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering. p. 643–653. FSE 2014, Association for Computing Machinery, New York, NY, USA (2014). <https://doi.org/10.1145/2635868.2635920>
21. Parry, O., Kapfhammer, G.M., Hilton, M., McMin, P.: A survey of flaky tests. ACM Trans. Softw. Eng. Methodol. **31**(1) (oct 2021). <https://doi.org/10.1145/3476105>
22. Parry, O., Kapfhammer, G.M., Hilton, M., McMin, P.: Empirically evaluating flaky test detection techniques combining test case rerunning and machine learning models. Empirical Software Engineering **28**(3), 72 (2023)
23. Vahabzadeh, A., Fard, A.M., Mesbah, A.: An empirical study of bugs in test code. In: 2015 IEEE international conference on software maintenance and evolution (ICSME). pp. 101–110. IEEE (2015)
24. Valiant, L.G.: A theory of the learnable. Communications of the ACM **27**(11), 1134–1142 (1984)